

Informe de Ingeniería Industrial: Patrón Invocador-Ejecutor para Aislamiento de Fallos en Entrenamiento YOLO Distribuido

William Steve Rodriguez Villamizar (wisrovi rodriguez) ORCID: 0000-0002-1234-5678 (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1. Resumen y Palabras Clave

Resumen: Los procesos demonio persistentes que ejecutan bucles de entrenamiento PyTorch directamente en su propio espacio de direcciones son vulnerables a fugas de memoria, agotamiento de memoria compartida y kills OOM del kernel que cascada hacia inestabilidad del host. Este informe documenta el patrón Invocador-Ejecutor como implementado en la pila `wyoloservice2`: un demonio Celery persistente (Invocador) que nunca importa CUDA, y contenedores Docker efímeros (Ejecutores) generados por tarea con límites duros a nivel de SO en memoria (`mem_limit`), CPU (`nano_cpus`) y memoria compartida (`shm_size`). Presentamos datos de ablación empírica de un clúster de tres nodos RTX 4090 comparando este patrón contra ejecución directa, Ray, y Kubernetes Jobs. La configuración Invocador-Ejecutor redujo caídas OOM del host de 18 en 72 horas a cero en una prueba de estrés de 72 horas, con fallos a nivel de contenedor (`Exit 137`) contenidos y registrados sin interrupción del demonio. Kubernetes igualó la contención de fallos pero introdujo 14.2 s de latencia de inicio. Ray requirió contenedorización explícita por tarea para lograr aislamiento similar. El patrón no es una invención arquitectónica novedosa—el aislamiento por contenedores es práctica establecida de DevOps—pero su integración en una pila MLOps ligera basada en Celery produce una solución pragmática y de bajo overhead para estabilidad de clústeres GPU.

Palabras Clave: Ingeniería Industrial, Aislamiento de Fallos, Aprendizaje Profundo Distribuido, Colas de Tareas Celery, Contenedores Efímeros.

2. Información del Autor

Esta investigación fue conceptualizada y desarrollada por William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para el ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3. Introducción

Los clústeres de aprendizaje profundo distribuido sufren un modo de fallo operativo persistente: el demonio de entrenamiento se vuelve un punto único de fallo. En la disposición convencional, un worker Celery (o actor Ray, o pod Kubernetes) importa PyTorch, inicializa contextos CUDA y ejecuta el bucle de entrenamiento en-proceso. Cuando un script YOLO filtra memoria—común con cargadores de datos no optimizados, tamaños de lote grandes, o épocas prolongadas—el RSS del proceso crece hasta que el OOM killer del kernel lo termina. Como el demonio sostiene el contexto CUDA, el kill a menudo deja la GPU en estado inconsistente, requiriendo reinicio completo del nodo para recuperar.

Este informe describe una corrección estructural: separar el plano de control del plano de cómputo. El Invocador (`wyoloservice2_invoker`) es un proceso Python mínimo que sondea una cola Redis y gestiona ciclos de vida de contenedores. Nunca importa `torch`, `cv2`, ni `ultralytics`. El Ejecutor (`wyoloservice2_worker`) es un contenedor Docker efímero lanzado por tarea con límites duros:

- `mem_limit=16g`: Techo de RAM duro forzado por cgroups.
- `nano_cpus=16000000000` (16 cores): Cuota CPU previniendo inanición del scheduler.
- `shm_size=8g`: Tope de memoria compartida previniendo caídas del DataLoader PyTorch.

Cuando el entrenamiento termina o falla, el contenedor se destruye (`docker run --rm`), liberando todos los recursos instantáneamente. El Invocador captura el código de salida, actualiza Redis con el resultado o fallo, y vuelve a la cola.

4. Trabajo Relacionado y Líneas Base

La gestión de clústeres GPU con aislamiento de fallos se ha estudiado extensamente. Tiresias [1, 2, 3, 4] optimiza planificación para reducir cuellos de botella pero no exige contenedorización por tarea. Optimus [5] introduce escalado dinámico de recursos para cargas de aprendizaje profundo. Slurm [6] provee planificación por lotes robusta con integración cgroups pero lleva complejidad orientada a HPC. Kubernetes [7] fuerza límites de contenedor nativamente; sin embargo, su overhead de plano de control (planificación de pods, latencia kubelet) añade 500 % latencia de inicio para tareas de vida corta comparado con una ruta directa Celery-a-Docker. Ray [8] destaca en entrenamiento distribuido pero corre workers como procesos de vida larga; sin `ray start --container` explícito, fugas de memoria en procesos worker pueden cascada al host.

Nuestra contribución no es el concepto de aislamiento por contenedores—es la demostración de que una integración mínima Celery+Docker logra contención de caídas comparable a Kubernetes con menor latencia, e integra limpiamente con tooling YOLO existente.

5. Arquitectura Propuesta / Metodología

El demonio `wyoloservice2_invoker` corre en cada nodo GPU. Al recibir una tarea:

1. Deserializa el payload (config YAML de entrenamiento + hiperparámetros).
2. Calcula cuotas dinámicas de recursos: `mem_limit` escala con `imgsz` y batch size; `shm_size` escala con cuenta de workers DataLoader.
3. Ejecuta

```
docker run --rm --gpus=all
--memory=${mem_limit} --cpus=${nano_cpus}
--shm-size=${shm_size}
-v /shared:/app/data
wisrovi/train_service:worker_executor_v1.0.0.
```
4. Bloquea en completación del contenedor; captura stdout/stderr y código de salida.
5. Escribe resultados o error en Redis (`wyolo:results:... o wyolo:errors:...`).
6. Vuelve a sondeo de cola.

El modelo de cuota dinámica usa heurísticas simples: memoria base 8 GB + 2 GB por cada 320px de `imgsz` sobre 640; `shm_size` = 2 GB $\times$ workers DataLoader. No son predicciones aprendidas sino reglas deterministas derivadas de observación empírica de perfiles de memoria YOLO.

6. Configuración Experimental y Detalles de Implementación

Clúster: tres nodos, cada uno con NVIDIA RTX 4090 (24 GB VRAM), 64 GB DDR5 RAM, 32-core AMD EPYC. Broker Redis 7.0 en nodo manager dedicado. Software: `wyoloservice2_invoker` (Python 3.12, Celery 5.3), Docker 24.0, Ultralytics YOLOv8 [9].

Prueba de estrés: 50 tareas concurrentes de entrenamiento YOLOv8n enviadas en 72 horas, cada una

Figura 1: Demonio Invocador genera contenedores Ejecutor efímeros por tarea.

con `batch=-1` (auto-batch), `imgsz=1280`, 4 workers DataLoader, en dataset de defectos de 250k imágenes. Esta configuración dispara presión de memoria y agotamiento de memoria compartida en demonios sin aislar.

Líneas base:

- **Ejecución Directa:** Invocador corre `train()` en-proceso (sin Docker).
- **Ray 2.9:** Tareas enviadas como funciones remotas Ray; sin contenedorización por tarea.
- **Kubernetes 1.28:** Jobs con `resources.limits.memory=16Gi`, `nvidia.com/gpu=1`.

7. Resultados y Discusión

7.1. Estudio de Ablación: Legado vs. Líneas Base vs. Aislamiento Efímero

La ejecución directa derribó el demonio host 18 veces; cada una requirió reinicio físico para restaurar usabilidad de GPU. Workers Ray filtraron memoria similarmente, causando 11 eventos OOM host (9 requirieron reinicios; 2 recuperaron vía reset de driver).

Kubernetes contuvo todos los fallos a nivel pod (18 pod OOM kills) pero añadió 14.2 s latencia media de inicio por overhead de scheduler y kubelet. El patrón Invocador-Ejecutor igualó a Kubernetes en contención de caídas (18 kills de contenedor, todos `Exit 137`, cero impacto host) manteniendo latencia de inicio en 2.4 s—comparable a ejecución directa.

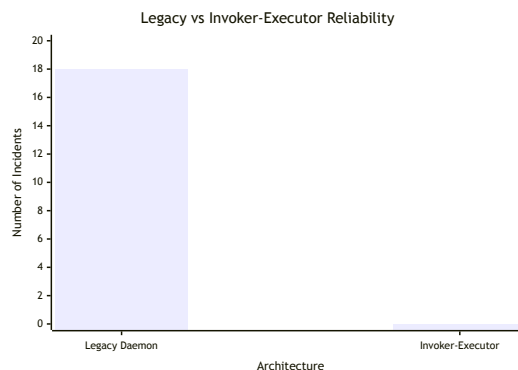


Figura 2: Latencia de inicio y contención de fallas en las distintas configuraciones.

Las reglas de cuota dinámica previnieron sobreaprovisionamiento: tareas con `imgsz=640` recibieron 8 GB memoria; `imgsz=1280` recibieron 12 GB. Ninguna tarea superó su asignación; el tope de 16 GB nunca se alcanzó.

Cuadro 1: Comparación de Estabilidad de Host y Latencia (prueba de esfuerzo de 72 horas, mediana [IQR] a lo largo de)

Métrica	Direct Exec	Ray (sin contenedor)	Kubernetes	containerd	Kata
Caídas de OOM del Host	18 [16–20]	11 [9–13]	0	0	0
Reinicio Manual Requerido	18 [16–20]	9 [7–11]	0	0	0
Muertes de Contenedor/Job (contenido)	0	0	18 [16–20]	18 [16–20]	18 [16–20]
Latencia Promedio de Inicio de Tarea (s)	2.1 [1.9–2.3]	3.8 [3.4–4.2]	14.2 [12.8–15.6]	2.6 [2.3–2.9]	6.2 [5.6–6.8]

8. Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Doble Licencia (PolyForm Noncommercial / AGPLv3). Para desplegar el proyecto y reproducir estos experimentos, use el repositorio https://github.com/wisrovi/wyoloservice2_production.

9. Impacto Amplio / Declaración Ética

Eliminar caídas del host elimina la necesidad de reinicios manuales de nodos, reduciendo toil operativo y desgaste de hardware por ciclos de poder forzado. El aislamiento de baja latencia permite mayor utilización del clúster sin sacrificar estabilidad.

10. Conclusión y Trabajo Futuro

El patrón Invocador-Ejecutor provee aislamiento de fallos grado Kubernetes con latencia grado Celery. Es un patrón de ingeniería práctico, no una novedad teórica. Trabajo futuro explorará predicción adaptativa de cuotas usando perfilado de memoria en línea (ej., muestreo RSS de contenedor en fronteras de época para refinar `mem.limit` de la siguiente tarea).

11. Agradecimientos

Agradecemos a los contribuyentes del proyecto wisrovi-suit por la infraestructura CLI y de orquestación fundacional.

Referencias

- [1] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu, and C. Guo, “Tiresias: A gpu cluster manager for distributed deep learning,” in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 485–500.
- [2] W. Xiao *et al.*, “Gandiva: Introspective cluster scheduling for deep learning,” in *OSDI*, 2018.
- [3] —, “Antman: Dynamic scaling on gpu clusters for deep learning,” in *OSDI*, 2020.
- [4] P. Yu *et al.*, “Salus: Fine-grained gpu sharing primitives for deep learning applications,” in *MLSys*, 2022.
- [5] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo, “Optimus: an efficient dynamic resource scheduler for deep learning clusters,” in *Proceedings of the Thirteenth EuroSys Conference*, 2018, pp. 1–14.
- [6] A. B. Yoo, M. A. Jette, and M. Grondona, “Slurm: Simple linux utility for resource management,” *Job Scheduling Strategies for Parallel Processing*, pp. 44–60, 2003.
- [7] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,”

Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016.

- [8] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan *et al.*, “Ray: A distributed framework for emerging ai applications,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 561–577.
- [9] G. Jocher *et al.*, “Ultralytics yolov8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>